

Nome: Hugo Leonardo Perestrelo Pires dos Santos

Turma: 2DSM

Professor: JULIANA PASQUINI

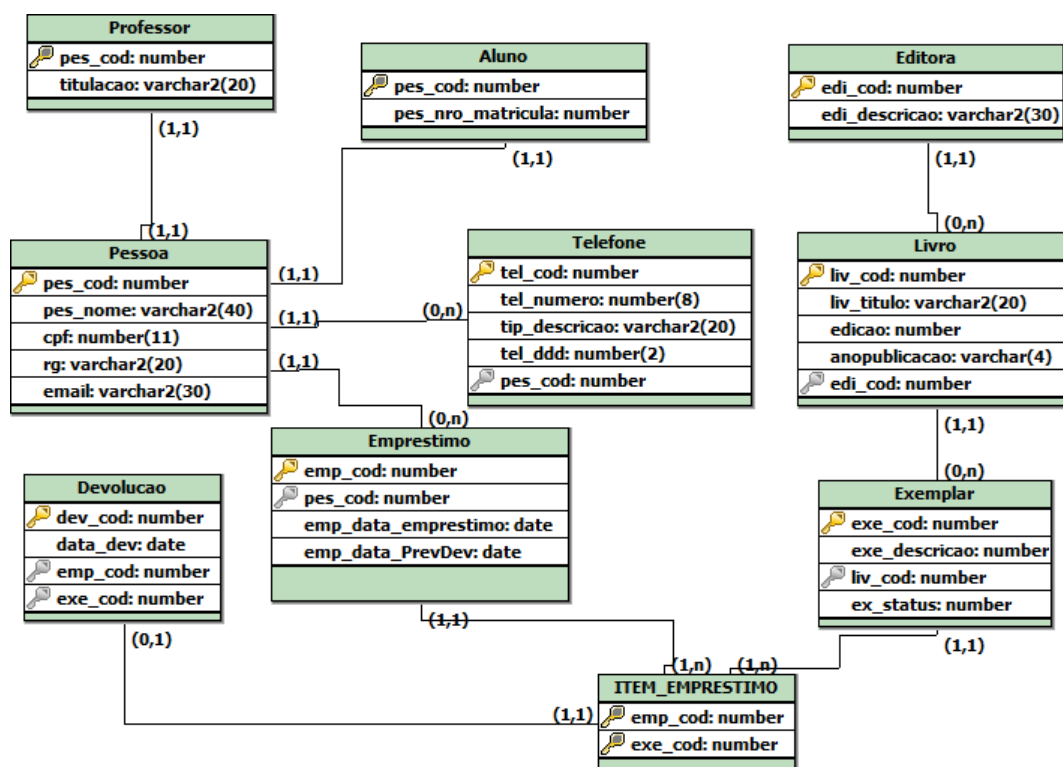
Data: 9/4/2026

PRÁTICA 5

INSTRUÇÕES:

- ✓ A prática pode ser resolvida em dupla.
- ✓ Os scripts estão prontos ambiente_biblioteca.zip.
- ✓ Utilizar o MYSQL
- ✓ Entregar o código SQL juntamente com os prints dos testes e resultados.
- ✓ Entregar no teams até: 23/4/2026 as consultas sql e os scripts alterados.

Estudo de Caso: Biblioteca Modelo E-R da Biblioteca



OBS: Os livros podem ter devoluções parciais, ou seja, um aluno pode pegar emprestado 3 livros diferentes e devolver apenas um.

- 1) Criar uma procedure que insere um novo livro na tabela Livro e automaticamente cadastra um número definido de exemplares para ele.

```

CREATE PROCEDURE CadastrarLivroComExemplares (
    IN p_titulo VARCHAR(20),
    IN p_edicao INT,
    IN p_ano VARCHAR(4),

```

Nome:Hugo Leonardo Perestrelo Pires dos Santos

Turma:2DSM

Professor: JULIANA PASQUINI

Data:9/4/2026

IN p_editora INT,
IN p_qtd_exemplares INT
)

R: delimiter \$\$

```
CREATE PROCEDURE  
CadastrarLivroComExemplares
```

```
( IN p_titulo VARCHAR(20),
```

```
IN p_edicao INT,
```

```
IN p_ano VARCHAR(4),
```

```
IN p_editora INT,
```

```
IN p_qtd_exemplares INT)
```

```
begin
```

```
declare i int default 1;
```

```
declare id int ;
```

```
insert into
```

```
Livro(liv_titulo,edicao,anopublicacao,edi_cod)v
```

```
a lues(p_titulo,p_edicao,p_ano,p_editora);
```

```
set id=LAST_INSERT_ID();
```

```
while i<p_qtd_exemplares
```

```
do insert into
```

```
Exemplar(exe_descricao,liv_cod)values(i,id)
```

```
; set i=i+1;
```

```
end
```

```
WHILE;
```

```
end $$;
```

```
;
```

```
delimiter ;
```

```
255 • CALL CadastrarLivroComExemplares('Algoritmos', 1, '2023', 1, 3);
```

```
256 • SELECT * FROM Livro WHERE liv_titulo = 'Algoritmos';
```

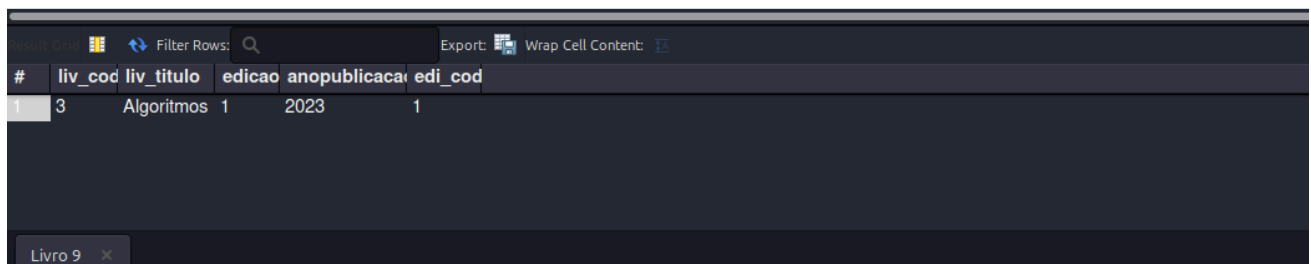
```
257 • SELECT * FROM Exemplar WHERE liv_cod = (SELECT liv_cod FROM Livro WHERE liv_titulo = 'Algoritmos');
```

```
258
```

```
259
```

```
260
```

```
261
```



#	liv_cod	liv_titulo	edicao	anopublicacao	edi_cod
1	3	Algoritmos	1	2023	1

- 2) Criar uma procedure que registre um empréstimo. CREATE PROCEDURE RegistrarEmprestimo (
- ```
IN p_pes_cod INT,
IN p_data_emprestimo DATE,
```

```
 IN p_data_prev_dev DATE
)
R:delimiter $$
CREATE PROCEDURE RegistrarEmprestimo (
 IN p_pes_cod INT,
 IN p_data_emprestimo DATE,
 IN p_data_prev_dev DATE
)
begin
insert into Emprestimo(pes_cod,emp_data_emprestimo,emp_data_PrevDev)
values(p_pes_cod,p_data_emprestimo,p_data_prev_dev);
```

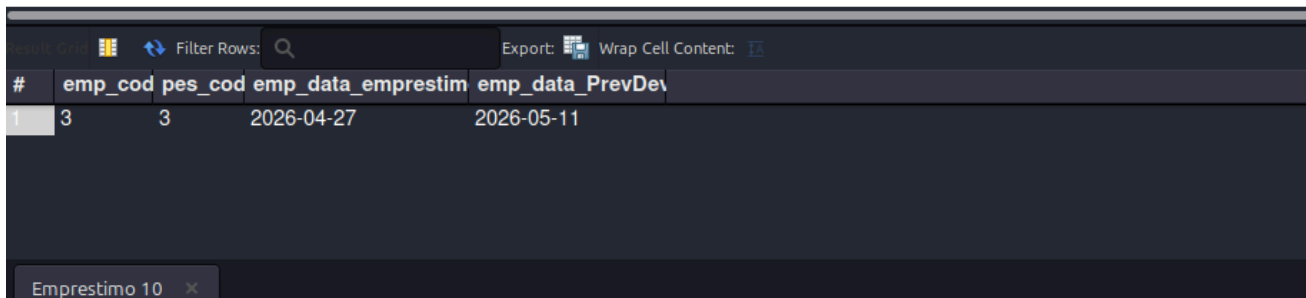
end \$\$ ;

delimiter ;

259

260 • CALL RegistrarEmprestimo(3, CURDATE(), DATE\_ADD(CURDATE(), INTERVAL 14 DAY));

261 • SELECT \* FROM Emprestimo WHERE pes\_cod = 3;



| # | emp_cod | pes_cod | emp_data_emprestimo | emp_data_PrevDev |
|---|---------|---------|---------------------|------------------|
| 1 | 3       | 3       | 2026-04-27          | 2026-05-11       |

- 3) Criar uma procedure que recebe o código do empréstimo e do exemplar e registra a devolução na tabela Devolucao (data atual).

```
CREATE PROCEDURE RegistrarDevolucao (
 IN p_emp_cod INT,
 IN p_exe_cod INT
)
```

R:delimiter \$\$

```
CREATE PROCEDURE RegistrarDevolucao (
 IN p_emp_cod INT,
 IN p_exe_cod INT
)
```

begin

```
insert into Devolucao(emp_cod,exe_cod) values(p_emp_cod,p_exe_cod);
```

end \$\$

delimiter ;

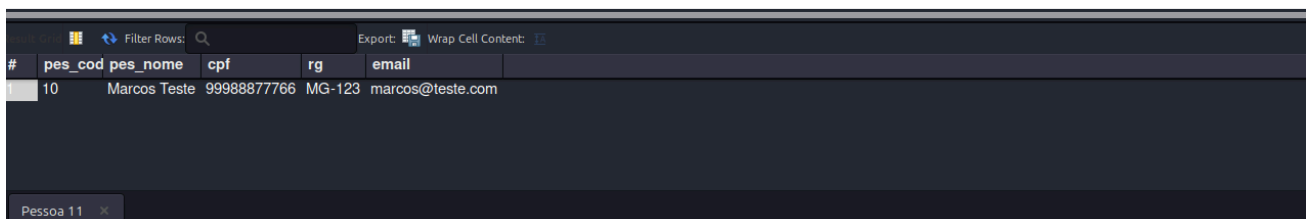
263

264 • CALL InserirPessoaComTelefone(10, 'Marcos Teste', 99988877766, 'MG-123', 'marcos@teste.com', 988884444, 'Celular', 31);

265 • SELECT \* FROM Pessoa WHERE pes\_cod = 10;

266 • SELECT \* FROM Telefone WHERE pes\_cod = 10;

267



| #  | pes_cod | pes_nome     | cpf         | rg     | email            |
|----|---------|--------------|-------------|--------|------------------|
| 10 | 10      | Marcos Teste | 99988877766 | MG-123 | marcos@teste.com |

- 4) Criar uma procedure que liste todos os empréstimos (e seus itens) realizados por uma pessoa com base em seu pes\_cod.

```
CREATE PROCEDURE HistoricoEmprestimos (
 IN p_pes_cod INT
)
```

R:delimiter \$\$

-- Criar uma procedure que liste todos os empréstimos (e seus itens) realizados por uma pessoa com  
-- base em seu pes\_cod.

```
CREATE PROCEDURE HistoricoEmprestimos (
 IN p_pes_cod INT
)
```

begin

```
select e.emp_data_emprestimo,e.emp_data_PrevDev,l.liv_titulo from Emprestimo e
inner join ITEM_EMPRESTIMO i
```

```

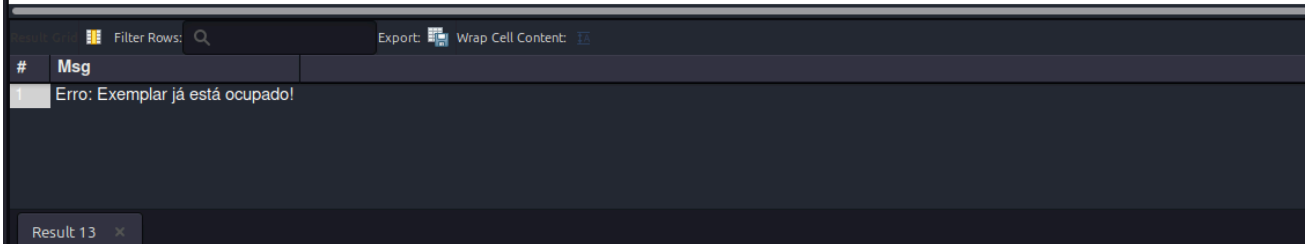
on
i.emp_cod=e.emp_cod
inner join Exemplar ex
on ex.exe_cod=i.exe_cod
inner join Livro l
on l.liv_cod=ex.liv_cod
where e.pes_cod=p_pes_cod;
end $$
delimiter ;

```

```

268
269 • CALL CriarEmprestimoSeguro(100, 10, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), 2);
270
271 • CALL CriarEmprestimoSeguro(101, 1, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 7 DAY), 2);
272

```



- 5) Criar uma procedure que retorna todos os livros e exemplares atualmente emprestados e ainda não devolvidos.

```

CREATE PROCEDURE LivrosEmprestadosNaoDevolvidos ()
R:delimiter $$
CREATE PROCEDURE LivrosEmprestadosNaoDevolvidos ()
begin

```

```

select l.liv_titulo,exe.exe_cod from Livro l
inner join Exemplar exe
on exe.liv_cod=l.liv_cod
inner join ITEM_EMPRESTIMO i
on i.exe_cod=exe.exe_cod
inner join Emprestimo e
on
e.emp_cod=i.emp_cod
where i.exe_cod NOT IN (SELECT exe_cod FROM Devolucao WHERE emp_cod = e.emp_cod);
end $$

```

delimiter ;

273

274 • **CALL** AtualizarSatus();

275 • **SELECT** exe\_cod, ex\_status **FROM** Exemplar;

276

| # | exe_cod | ex_status |
|---|---------|-----------|
| 1 | 1       | 1         |
| 2 | 2       | 1         |
| 3 | 3       | 0         |
| 4 | 4       | 0         |
| 5 | 5       | 1         |
| 6 | 6       | 0         |
| 7 | 7       | 0         |
| 8 | 8       | 0         |

Exemplar 14

- 6) Criar um procedure para atualizar o campo ex\_status para 0, se o livro estiver disponível (não emprestados ou já devolvidos), caso contrário ex\_status deve ficar com 1.

```

CREATE PROCEDURE AtualizarSatus()
R:delimiter $$
CREATE PROCEDURE AtualizarSatus()
begin
update Exemplar exe set exe.ex_status= if(exe.exe_cod in
(select exe_cod from Devolucao) or exe.exe_cod not in (select exe_cod from
ITEM_EMPRESTIMO),0,1);
end $$

```

delimiter

277

278 • **CALL** DisponibilidadeLivro(1, @qtd);

279 • **SELECT** @qtd **AS** Exemplares\_Disponiveis\_Livro\_1;

| # | Exemplares_Disponiveis_Livro |
|---|------------------------------|
| 1 | 2                            |

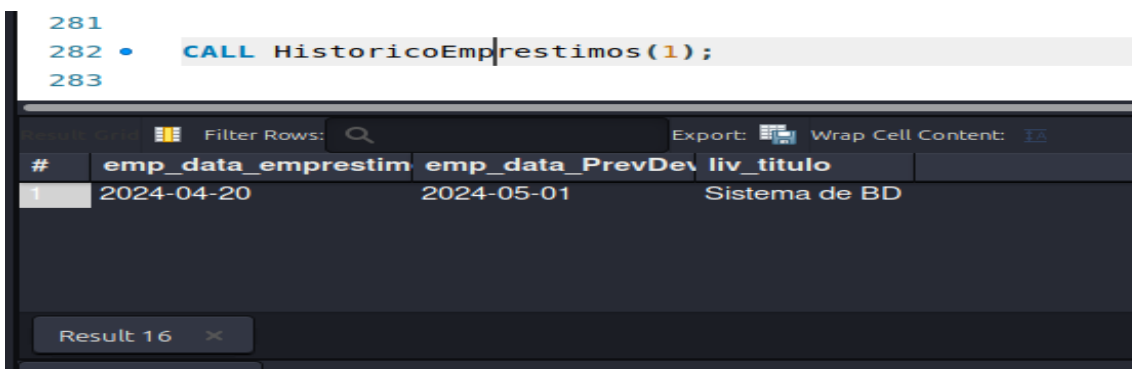
Result 15

;

7)

Criar uma procedure que, dado o liv\_cod, retorna a quantidade de exemplares disponíveis (não emprestados ou já devolvidos). OBS: É possível verificar pelo ex\_status.

```
CREATE PROCEDURE DisponibilidadeLivro (
 IN p_liv_cod INT, OUT p_quantidade int)
)
R:delimiter $$
CREATE PROCEDURE DisponibilidadeLivro (
 IN p_liv_cod INT, OUT p_quantidade int
)
begin
 select count(e.exe_cod) into p_quantidade from Exemplar e
 where e.liv_cod=p_liv_cod and e.ex_status=0;
end $$
delimiter
```



The screenshot shows a SQL IDE interface. The top part displays a SQL script with three lines: line 281 is empty, line 282 contains the call `CALL HistoricoEmprestimos(1);`, and line 283 is empty. Below the script, a table titled "Result 16" is displayed. The table has four columns: "#", "emp\_data\_emprestim", "emp\_data\_PrevDev", and "liv\_titulo". The first row of data shows the values "1", "2024-04-20", "2024-05-01", and "Sistema de BD".

| # | emp_data_emprestim | emp_data_PrevDev | liv_titulo    |
|---|--------------------|------------------|---------------|
| 1 | 2024-04-20         | 2024-05-01       | Sistema de BD |

;

8) Criar uma procedure que atualiza a titulação de um professor específico.



```

CREATE PROCEDURE AtualizarTitulacao (
 IN p_pes_cod INT,
 IN p_nova_titulacao VARCHAR(20)
)
R:delimiter $$
CREATE PROCEDURE AtualizarTitulacao (
 IN p_pes_cod INT,
 IN p_nova_titulacao VARCHAR(20)
)
begin
update Professor prof set prof.titulacao=p_nova_titulacao
where prof.pes_cod=p_pes_cod;
end $$
delimiter
;

```

| # | dev_cod | data_dev   | emp_cod | exe_cod |
|---|---------|------------|---------|---------|
| 1 | 1       | 2026-04-27 | 100     | 2       |

- 9) Criar uma procedure que insere uma nova pessoa na tabela Pessoa e adiciona um ou mais telefones associados.

```

CREATE PROCEDURE InserirPessoaComTelefone
(IN p_pes_cod INT,
 IN p_nome VARCHAR(40),
 IN p_cpf BIGINT,
 IN p_rg VARCHAR(20),
 IN p_email VARCHAR(30),
 IN p_tel_numero INT,
 IN p_tip_descricao VARCHAR(20),
 IN p_tel_ddd INT
)
R:delimiter $$

```

```

CREATE PROCEDURE InserirPessoaComTelefone
(IN p_pes_cod INT,
 IN p_nome VARCHAR(40),
 IN p_cpf BIGINT,
 IN p_rg VARCHAR(20),
 IN p_email VARCHAR(30),
 IN p_tel_numero INT,
 IN p_tip_descricao VARCHAR(20),
 IN p_tel_ddd INT
)

```

```

begin
insert into Pessoa(pes_cod,pes_nome,cpf,rg,email)values(p_pes_cod,p_nome,p_cpf,p_rg,p_email);
insert into Telefone(tel_numero,tip_descricao,tel_ddd,pes_cod)
values(p_tel_numero,p_tip_descricao,p_tel_ddd,p_pes_cod);
end $$
delimiter
;

```

| # | liv_titulo     | exe_cod |
|---|----------------|---------|
| 1 | Sistema de BD  | 1       |
| 2 | Banco de Dados | 5       |

- 10) Criar uma procedure que lista todos os empréstimos cuja data prevista de devolução já passou e ainda não têm devolução registrada.

```
CREATE PROCEDURE DevolucoesEmAtraso ()
```

```

R:delimiter $$
CREATE PROCEDURE DevolucoesEmAtraso ()
begin
select e.emp_cod,i.exe_cod from Emprestimo e
inner join ITEM_EMPRESTIMO i
on i.emp_cod=e.emp_cod
where e.emp_data_PrevDev<Date(Now()) and not exists (select 1 from Devolucao d where
i.emp_cod=d.emp_cod and d.exe_cod=i.exe_cod);
end $$

```

delimiter

```

291
292 • CALL AtualizarTitulacao(4, 'Pós-Doutorado');
293 • SELECT * FROM Professor WHERE pes_cod = 4;

```

| # | pes_cod | titulacao     |
|---|---------|---------------|
| 1 | 4       | Pós-Doutorado |

Professor 19

- 11) Criar uma procedure que recebe o código de um exemplar. Se ele já estiver emprestado e não devolvido, exiba uma mensagem de erro. Caso contrário, registre o empréstimo.

```

R:delimiter $$
create procedure CriarEmprestimoSeguro(in e_emp_cod int ,in p_pes_cod int,in e_emp_data_emprestimo
date,in e_emp_data_prevdev date,in e_exemplar int)
begin
if(select ex_status from Exemplar where e_exemplar=exe_cod)=0 then
insert into
Emprestimo(emp_cod,pes_cod,emp_data_emprestimo,emp_data_PrevDev)values(e_emp_cod,p_pes_cod,e_em
p_data_emprestimo,e_emp_data_prevdev);
insert into ITEM_EMPRESTIMO(exe_cod,emp_cod)values(e_exemplar,e_emp_cod);
UPDATE Exemplar SET ex_status = 1 WHERE exe_cod = e_exemplar;
else
select 'Erro ao inserir'
;
end if;
end $$
delimiter
;

```

```

295
296 • INSERT INTO Emprestimo (pes_cod, emp_data_emprestimo, emp_data_PrevDev) VALUES (2, '2020-01-01', '2020-01-10');
297 • SET @ultimo_emp = LAST_INSERT_ID();
298 • INSERT INTO ITEM_EMPRESTIMO VALUES (@ultimo_emp, 7);
299 • CALL DevolucoesEmAtraso();
300

```

| # | emp_cod | exe_cod |
|---|---------|---------|
| 1 | 1       | 1       |
| 2 | 2       | 5       |
| 3 | 101     | 7       |

Result 20

- 12) Criar uma procedure que receba o pes\_cod. Verifique se essa pessoa tem empréstimos. Se tiver, **não permitir exclusão**. Caso contrário, exclua da tabela Pessoa.

R:delimiter \$\$

```

create procedure VerEmp(in p_pes_cod int)
begin
if(exists(select 1 from Emprestimo where pes_cod=p_pes_cod)) then
select 'Não Podemos escluir, pois a pessoa possui Emprestimos';
else
delete from Pessoa where pes_cod=p_pes_cod ;
end if;
end $$
delimiter
;

```

```

301
302 • CALL Atualiza7d(2);
303 • SELECT * FROM Emprestimo WHERE pes_cod = 2;
304

```

| # | emp_cod | pes_cod | emp_data_emprestim | emp_data_PrevDev |
|---|---------|---------|--------------------|------------------|
| 1 | 2       | 2       | 2024-04-22         | 2024-05-12       |
| 2 | 101     | 2       | 2020-01-01         | 2020-01-17       |

13)

Criar uma procedure que percorre todos os empréstimos de uma pessoa e, se ainda não devolvido, **atualiza a data prevista de devolução para +7 dias**.

R:delimiter \$\$

```

create procedure Atualiza7d(in p_pes_cod int)
begin
declare e_emp_cod int;
declare dataEmp date;

```

